МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
„КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ”
НАВЧАЛЬНО-НАУКОВИЙ КОМПЛЕКС
„ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ”

ЛАБОРАТОРНА РОБОТА №3

з курсу: „ Методи та системи штучного інтелекту ”

виконав студент 4го курсу
групи ДА-11
Сидько Н.С.

КИЇВ
2024

Мета роботи: Здобути навички програмної реалізації машинного навчання на базі нейронних мереж.

Завдання: Дослідити методологію машинного навчання в задачі класифікації візуальних образів за допомогою нейронної мережі Хопфілда на прикладі чорно-білих образів літер абетки.

Варіант

№	Літери
3	В М Я

Для реалізації данної лабораторної роботи, оберему мову програмування Python, так як він досить зручний у використанні через наявність всіх необхідних бібліотек для реалізації мережі Хопфілда. Також у ньому наявна потужна та проста бібліотека для роботи з інтерфейсом програми

Мережа Хопфілда — це повнозв’язана нейронна мережа, яка працює як асоціативна пам'ять для збереження і відновлення образів. Вона складається з одного шару нейронів, які взаємодіють між собою через симетричну матрицю ваг, що розраховується за правилом Хебба. У процесі роботи мережа зводиться до стабільного стану (рівноваги), що відповідає одному із запам’ятованих патернів. Вхідний сигнал (наприклад, спотворений образ) поступово обробляється шляхом ітеративного оновлення станів нейронів, мінімізуючи енергетичну функцію мережі. Така модель використовується для задач розпізнавання образів, фільтрації шумів і оптимізації.

Реалізація мережі

Ініціалізація

```
def __init__(self, size):
    self.size = size
    self.weights = np.zeros((size, size))
```

Ініціалізує мережу, задаючи розмір та нульову матрицю ваг. Визначає кількість нейронів і створює матрицю ваг, яка буде оновлюватись під час навчання.

Навчання

```
def train_patterns(self, patterns):  2 usages
    self.weights = np.zeros((self.size, self.size))  # Скидання ваг
    for pattern in patterns:
        pattern = pattern.reshape(self.size, 1)
        self.weights += np.dot(pattern, pattern.T)
    np.fill_diagonal(self.weights, val: 0)
```

Приймає патерни для навчання та формує симетричну матрицю ваг на основі правила Хебба

Розпізнавання паттерна

```
    for _ in range(max_iterations):
        new_pattern = sgn(np.dot(self.weights, pattern))
        if np.array_equal(new_pattern, pattern):
            break
        pattern = new_pattern
    return pattern
```

Використовується для розпізнавання, починаючи з вхідного образу, мережа ітеративно змінює стани нейронів до досягнення стабільного стану

Перетворення образів

```
def preproc_pattern(grid):  3 usages
    return np.where(np.array(grid).flatten() > 0, 1, -1)
```

Перетворює образ (матрицю) в одномірний вектор з елементами +1 і -1.

Візуалізація

```
def draw_training_sample_with_label(frame, grid, size, label_text):  2 usages

    sample_frame = tk.Frame(frame)
    sample_frame.pack(side=tk.LEFT, padx=5)

    canvas = tk.Canvas(sample_frame, width=size * 30, height=size * 30, bg="whit
    canvas.pack()

    for i in range(size):
        for j in range(size):
            color = "black" if grid[i][j] == 1 else "white"
            canvas.create_rectangle(
                j * 30, i * 30, (j + 1) * 30, (i + 1) * 30,
                fill=color, outline="black"
            )

    label = tk.Label(sample_frame, text=label_text, font=("Helvetica", 10))
    label.pack()
```

Основний цикл

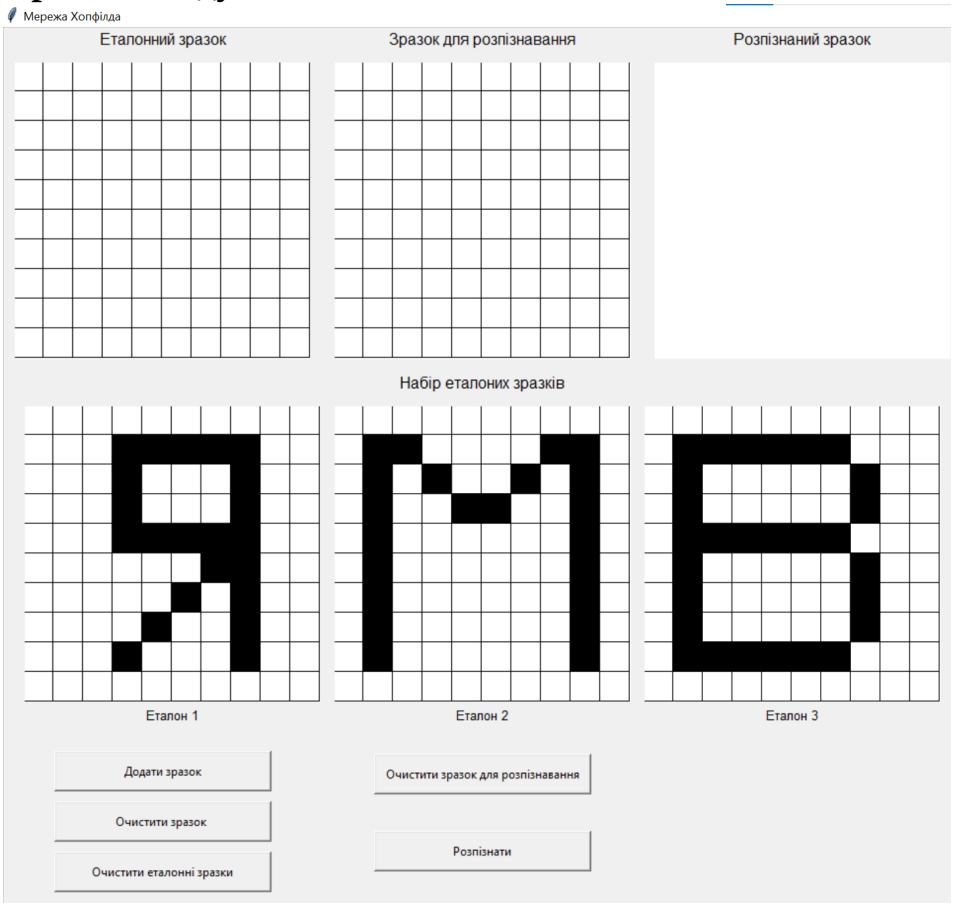
Функція main - створює графічний інтерфейс і налаштовує всі компоненти: (Поля для тренувальних і розпізнаваних зразків, кнопки для управління. візуалізацію результатів)

```
def main():  1 usage
    global size, hopfield_net, patterns, training_grid, recognizing_gri

    root = tk.Tk()
    root.title("Мережа Хопфілда")

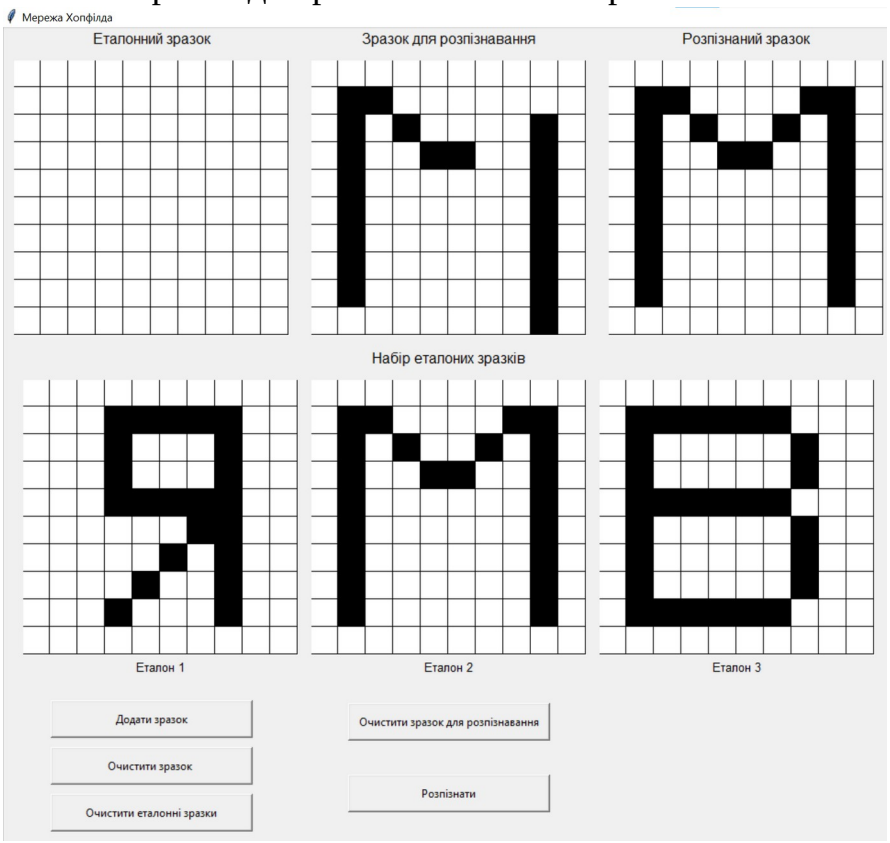
    size = 10
    hopfield_net = HopfieldNetwork(size * size)
    patterns = []
```

Результат роботи коду:

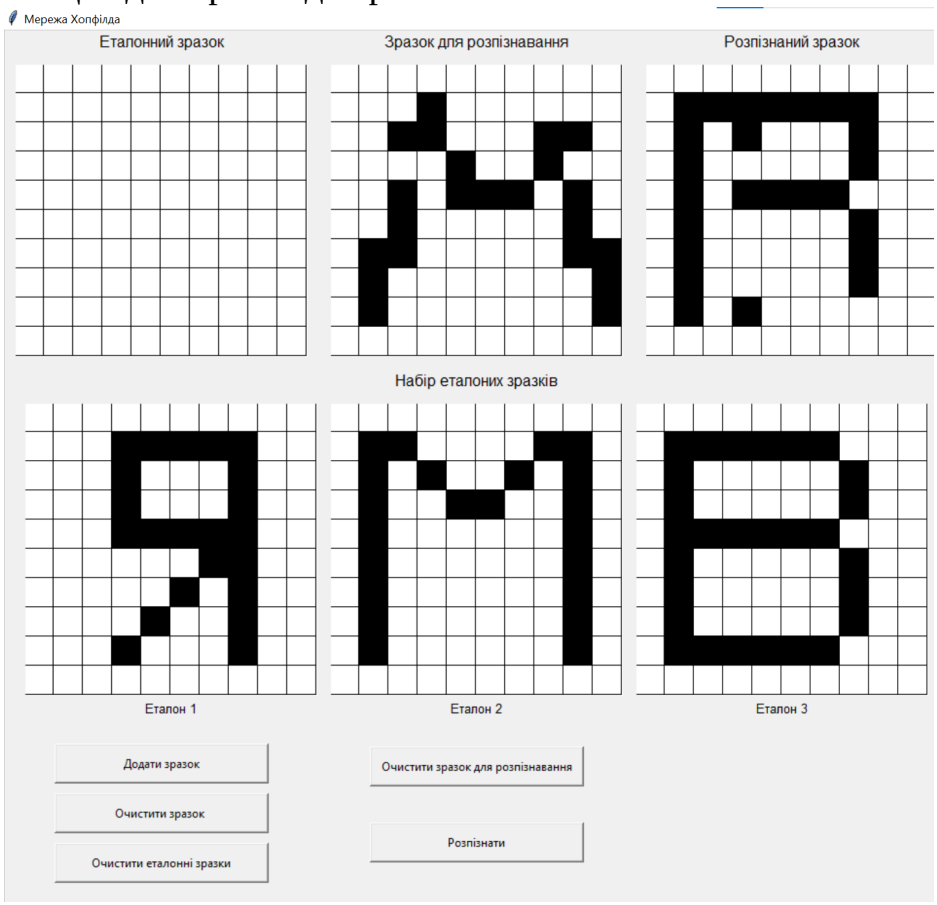


При запуску прогарми згідно варіанту у нас вже прописані необхідні літери (тренувальні зразки – еталони)

Намалюємо якийсь зразок для розпізнавання літери:



Намалюємо ще один зразок для розпізнавання:



Бачимо що в нас виникла химера, тобто це стан, у якому мережа не може стабілізуватися на одному з запам'ятованих образів і натомість формує комбінацію кількох образів. Це відбувається через надмірну кількість збережених патернів, їхню схожість або сильний шум у вхідних даних

Висновок.

У ході виконання лабораторної роботи №3 було написано реалізацію нейронної мережі Хопфілда для класифікації чорно-білих візуальних образів. Мережа хопфілда була успішно протестована, за допомогою розпізнавання літер Я М В згідно варіанту завдання. При значному пошкодженні зразка для розпізнання у нас виник химерний стан (поєднання еталонних образів)